

Guía completa — Simon Says con ATmega328P + PIC16F887

Visión general del sistema

Antes de entrar en el detalle de cada parte, aquí tienes el diagrama de arquitectura del sistema completo:

V

visualize

☐

v

visualize show_widget

☐

Parte 1 — Diseño del juego

Mecánica Simon Says adaptada:

El ATmega genera una secuencia aleatoria de colores (1–N). La muestra en la matriz LED uno por uno, enviando simultáneamente el comando al PIC para que toque la nota correspondiente. Luego el jugador debe repetir la secuencia con los 4 pulsadores. Si acierta, la secuencia crece en uno. Si falla, se muestra una animación de error y se reinicia.

Mapa de colores → símbolo LED → nota:

Color	Símbolo en matriz	Nota (Hz)	Pin botón
Verde	Cuadrante inf-izq lleno	262 Hz (Do)	PD2
Rojo	Cuadrante sup-izq lleno	330 Hz (Mi)	PD3
Azul	Cuadrante sup-der lleno	392 Hz (Sol)	PD4
Amarillo	Cuadrante inf-der lleno	523 Hz (Do alto)	PD5

Niveles:

- Nivel 1 (fácil): secuencia máx. 5 colores, delay 800 ms entre cada uno
 - Nivel 2 (medio): secuencia máx. 8 colores, delay 500 ms
 - Nivel 3 (difícil): secuencia máx. 12 colores, delay 300 ms, tiempo de respuesta limitado a 3 s
-

Parte 2 — Esquema de comunicación entre microcontroladores

El enunciado dice "conexión paralela". Usaremos 4 líneas digitales simples (lo más directo posible en C nativo):

- 3 bits de datos (PC0, PC1, PC2 del ATmega → RB0, RB1, RB2 del PIC): codifican el comando
- 1 línea strobe (PC3 del ATmega → RB3 del PIC): pulso que indica al PIC que lea el bus

Tabla de comandos:

Comando	PC2	PC1	PC0	Significado
0	0	0	0	Silencio / idle
1	0	0	1	Toca nota verde
2	0	1	0	Toca nota roja
3	0	1	1	Toca nota azul
4	1	0	0	Toca nota amarilla
5	1	0	1	Melodía inicio
6	1	1	0	Melodía victoria
7	1	1	1	Melodía error

Parte 3 — Circuito en Proteus

Conexiones ATmega328P:

Matriz LED (MAX7219 vía SPI):

PB2 (SS) → CS del MAX7219

PB3 (MOSI) → DIN del MAX7219

PB5 (SCK) → CLK del MAX7219

Pulsadores (con pull-up interno):

PD2 → Botón Verde

PD3 → Botón Rojo

PD4 → Botón Azul

PD5 → Botón Amarillo

Bus paralelo hacia PIC:

PC0 → RB0 (PIC) — bit dato 0

PC1 → RB1 (PIC) — bit dato 1

PC2 → RB2 (PIC) — bit dato 2

PC3 → RB3 (PIC) — strobe

Cristal: 16 MHz externo + caps 22pF

AVCC → VCC (5V) con cap 100nF

Conexiones PIC16F887:

Bus paralelo desde ATmega:

RB0 → PC0 (ATmega) — bit dato 0

RB1 → PC1 (ATmega) — bit dato 1

RB2 → PC2 (ATmega) — bit dato 2

RB3 → PC3 (ATmega) — strobe (interrupción INT0 o polling)

Buzzer:
RC2 (CCP1 / PWM) → Buzzer → GND
(resistor 100Ω en serie)

Cristal: 8 MHz externo
MCLR → VCC vía resistor 10kΩ

Tip Proteus: Usa el componente MAX7219 directamente disponible en la librería. Para el buzzer usa BUZZER o SOUNDER. Los pulsadores son BUTTON. Configura el ATmega con CKSEL=1111 (cristal externo 16 MHz) en las fuses del componente.

Parte 4 — Código ATmega328P (C nativo, PlatformIO)

```
C
// ATmega328P — Simon Says
// PlatformIO, framework: avr, board: uno
// main.c

#include <avr/io.h>
#include <avr/interrupt.h>
#include <util/delay.h>
#include <stdlib.h>

// — Pines —————
#define BTN_GREEN PD2
#define BTN_RED PD3
#define BTN_BLUE PD4
#define BTN_YELLOW PD5

#define COM_D0 PC0 // bus paralelo datos
#define COM_D1 PC1
#define COM_D2 PC2
#define COM_STB PC3 // strobe

// MAX7219 SPI (manual bitbang — más claro en C nativo)
#define MAX_CS PB2
#define MAX_DIN PB3
#define MAX_CLK PB5

// — Registros MAX7219 —————
#define REG_DECODE 0x09
#define REG_INTENSITY 0x0A
#define REG_SCANLIMIT 0x0B
#define REG_SHUTDOWN 0x0C
#define REG_TEST 0x0F

// — Colores / comandos —————
#define COLOR_GREEN 1
#define COLOR_RED 2
#define COLOR_BLUE 3
#define COLOR_YELLOW 4
#define CMD_MELODY_START 5
#define CMD_MELODY_WIN 6
#define CMD_MELODY_FAIL 7
```

```

#define CMD_SILENCE    0

// — Niveles —————
#define LVL1_MAX  5
#define LVL2_MAX  8
#define LVL3_MAX 12
#define LVL1_DLY 800
#define LVL2_DLY 500
#define LVL3_DLY 300

// — Patrones LED por cuadrante (8 filas, bit=columna) —
// Verde: cuadrante inferior-izquierdo (filas 4-7, cols 0-3)
const uint8_t pat_green[8] = {0x00,0x00,0x00,0x00,0x0F,0x0F,0x0F,0x0F};
// Rojo: cuadrante superior-izquierdo (filas 0-3, cols 0-3)
const uint8_t pat_red[8]  = {0x0F,0x0F,0x0F,0x0F,0x00,0x00,0x00,0x00};
// Azul: cuadrante superior-derecho (filas 0-3, cols 4-7)
const uint8_t pat_blue[8] = {0xF0,0xF0,0xF0,0xF0,0x00,0x00,0x00,0x00};
// Amarillo: cuadrante inferior-derecho (filas 4-7, cols 4-7)
const uint8_t pat_yellow[8] = {0x00,0x00,0x00,0x00,0xF0,0xF0,0xF0,0xF0};
// Error: X en toda la pantalla
const uint8_t pat_error[8] = {0x81,0x42,0x24,0x18,0x18,0x24,0x42,0x81};
// Win: carita feliz
const uint8_t pat_win[8]  = {0x3C,0x42,0xA5,0x81,0xA5,0x99,0x42,0x3C};
// Pantalla vacía
const uint8_t pat_blank[8] = {0,0,0,0,0,0,0,0};

// Letras L1, L2, L3 para mostrar nivel (simplificadas 8x8)
const uint8_t pat_L1[8] = {0x60,0x20,0x20,0x20,0x20,0x23,0x3F,0x00};
const uint8_t pat_L2[8] = {0x60,0x20,0x20,0x0E,0x10,0x30,0x3F,0x00};
const uint8_t pat_L3[8] = {0x60,0x20,0x20,0x0E,0x02,0x22,0x1C,0x00};

// — Variables globales —————
uint8_t sequence[LVL3_MAX];
uint8_t seq_len = 0;
uint8_t nivel  = 1;

// — MAX7219: funciones SPI bitbang —————
static void max_write_byte(uint8_t b) {
    for (int i = 7; i >= 0; i--) {
        if (b & (1 << i)) PORTB |= (1 << MAX_DIN);
        else             PORTB &= ~(1 << MAX_DIN);
        PORTB |= (1 << MAX_CLK);
        PORTB &= ~(1 << MAX_CLK);
    }
}

static void max_write(uint8_t reg, uint8_t data) {
    PORTB &= ~(1 << MAX_CS);
    max_write_byte(reg);
    max_write_byte(data);
    PORTB |= (1 << MAX_CS);
}

static void max_init(void) {
    max_write(REG_TEST, 0x00);
    max_write(REG_SHUTDOWN, 0x01); // normal operation
    max_write(REG_DECODE, 0x00); // sin decodificación BCD
    max_write(REG_SCANLIMIT, 0x07); // 8 dígitos
    max_write(REG_INTENSITY, 0x08); // brillo medio

```

```

}

static void max_display(const uint8_t *pat) {
    for (uint8_t row = 0; row < 8; row++)
        max_write(row + 1, pat[row]);
}

// — Comunicación con PIC —————
static void pic_send(uint8_t cmd) {
    // escribir 3 bits de datos
    if (cmd & 1) PORTC |= (1 << COM_D0); else PORTC &= ~(1 << COM_D0);
    if (cmd & 2) PORTC |= (1 << COM_D1); else PORTC &= ~(1 << COM_D1);
    if (cmd & 4) PORTC |= (1 << COM_D2); else PORTC &= ~(1 << COM_D2);
    _delay_us(10);           // bus settle
    PORTC |= (1 << COM_STB);    // strobe HIGH
    _delay_us(50);
    PORTC &= ~(1 << COM_STB);    // strobe LOW
}

// — Lectura de botones (retorna 0 si ninguno) —————
static uint8_t read_button(void) {
    if (!(PIND & (1 << BTN_GREEN))) return COLOR_GREEN;
    if (!(PIND & (1 << BTN_RED))) return COLOR_RED;
    if (!(PIND & (1 << BTN_BLUE))) return COLOR_BLUE;
    if (!(PIND & (1 << BTN_YELLOW))) return COLOR_YELLOW;
    return 0;
}

static void wait_button_release(void) {
    while (read_button()) _delay_ms(10);
}

// — Mostrar un color con sonido —————
static void show_color(uint8_t color, uint16_t dur_ms) {
    const uint8_t *pat;
    switch (color) {
        case COLOR_GREEN: pat = pat_green; break;
        case COLOR_RED: pat = pat_red; break;
        case COLOR_BLUE: pat = pat_blue; break;
        case COLOR_YELLOW: pat = pat_yellow; break;
        default: return;
    }
    pic_send(color);
    max_display(pat);
    _delay_ms(dur_ms);
    max_display(pat_blank);
    pic_send(CMD_SILENCE);
    _delay_ms(100);
}

// — Mostrar nivel brevemente —————
static void show_level(void) {
    const uint8_t *pat;
    switch (nivel) {
        case 1: pat = pat_L1; break;
        case 2: pat = pat_L2; break;
        default: pat = pat_L3; break;
    }
    max_display(pat);
}

```

```

    _delay_ms(1200);
    max_display(pat_blank);
    _delay_ms(300);
}

// — Animación de error —————
static void anim_error(void) {
    pic_send(CMD_MELODY_FAIL);
    for (uint8_t i = 0; i < 3; i++) {
        max_display(pat_error);
        _delay_ms(250);
        max_display(pat_blank);
        _delay_ms(150);
    }
    pic_send(CMD_SILENCE);
}

// — Animación de victoria —————
static void anim_win(void) {
    pic_send(CMD_MELODY_WIN);
    max_display(pat_win);
    _delay_ms(2000);
    max_display(pat_blank);
    pic_send(CMD_SILENCE);
}

// — Parámetros según nivel —————
static uint8_t nivel_max_seq(void) {
    if (nivel == 1) return LVL1_MAX;
    if (nivel == 2) return LVL2_MAX;
    return LVL3_MAX;
}
static uint16_t nivel_delay(void) {
    if (nivel == 1) return LVL1_DLY;
    if (nivel == 2) return LVL2_DLY;
    return LVL3_DLY;
}
static uint16_t nivel_timeout(void) {
    // timeout por pulsación (ms), nivel 3 más corto
    if (nivel == 1) return 8000;
    if (nivel == 2) return 5000;
    return 3000;
}

// — Lógica principal del juego —————
static void game_loop(void) {
    seq_len = 0;

    while (1) {
        // Agregar color aleatorio a la secuencia
        sequence[seq_len] = (rand() % 4) + 1;
        seq_len++;

        // Mostrar secuencia completa
        _delay_ms(600);
        for (uint8_t i = 0; i < seq_len; i++)
            show_color(sequence[i], nivel_delay());

        // Leer respuesta del jugador
    }
}

```

```

for (uint8_t i = 0; i < seq_len; i++) {
    uint16_t timeout = nivel_timeout();
    uint8_t pressed = 0;

    // Esperar pulsación con timeout
    while (timeout > 0) {
        pressed = read_button();
        if (pressed) break;
        _delay_ms(10);
        timeout -= 10;
    }

    if (!pressed) { anim_error(); return; } // timeout

    // Mostrar feedback visual del botón presionado
    show_color(pressed, 300);
    wait_button_release();
    _delay_ms(50);

    if (pressed != sequence[i]) { anim_error(); return; } // error
}

// Verificar victoria de nivel
if (seq_len >= nivel_max_seq()) {
    anim_win();
    // Subir de nivel si no está en 3
    if (nivel < 3) { nivel++; show_level(); }
    return; // reinicia el juego con nueva secuencia
}
}
}

// — main —————
int main(void) {
    // Configurar pines
    DDRB |= (1 << MAX_CS) | (1 << MAX_DIN) | (1 << MAX_CLK);
    DDRC |= (1 << COM_D0) | (1 << COM_D1) | (1 << COM_D2) | (1 << COM_STB);
    DDRD &= ~(1 << BTN_GREEN) | (1 << BTN_RED) | (1 << BTN_BLUE) | (1 << BTN_YELLOW);
    PORTD |= (1 << BTN_GREEN) | (1 << BTN_RED) | (1 << BTN_BLUE) | (1 << BTN_YELLOW); // pull-ups

    PORTB |= (1 << MAX_CS); // CS inactivo
    PORTC = 0x00;

    max_init();
    srand(42); // semilla fija (en producción usar ADC flotante)

    // Inicio
    pic_send(CMD_MELODY_START);
    show_level();
    _delay_ms(500);

    while (1) {
        game_loop();
        _delay_ms(1000);
        show_level();
    }
}

```

platformio.ini:

```
ini
[env:uno]
platform = atmelavr
board = uno
framework = avr
build_flags = -DF_CPU=16000000UL
```

Parte 5 — Código PIC16F887 (C en MikroC for PIC)

```
C
// PIC16F887 — Receptor de comandos y reproductor de melodías
// MikroC for PIC, oscilador externo 8 MHz

// Configuración de bits (MikroC pragma)
// Config: XT oscillator, WDT off, MCLR on, LVP off
// Bits de config en MikroC: Project > Edit Project > Device Flags

// — Pines del bus paralelo (PORTB) —————
// RB0 = D0, RB1 = D1, RB2 = D2, RB3 = STROBE
#define STROBE_PIN RB3_bit

// — Buzzer en RC2 (CCP1, PWM Timer2) —————
// MikroC tiene librería Sound que usa RC2 automáticamente

// — Frecuencias de cada color (Hz) —————
#define FREQ_GREEN 262u // Do4
#define FREQ_RED 330u // Mi4
#define FREQ_BLUE 392u // Sol4
#define FREQ_YELLOW 523u // Do5

// — Duración nota simple (ms) —————
#define NOTE_DUR 400u

// — Prototipos —————
void play_note(unsigned int freq, unsigned int dur_ms);
void melody_start(void);
void melody_win(void);
void melody_fail(void);
void process_command(unsigned char cmd);

// — Leer bus: 3 bits de datos —————
unsigned char read_bus(void) {
    return (unsigned char)(PORTB & 0x07); // máscara bits 0-2
}

// — Tocar nota usando Sound_Play de MikroC —————
void play_note(unsigned int freq, unsigned int dur_ms) {
    Sound_Play(freq, dur_ms);
}

// — Melodías de evento —————
void melody_start(void) {
    // Escala ascendente rápida
    play_note(262, 120);
    play_note(330, 120);
    play_note(392, 120);
    play_note(523, 200);
}
```



```

}

void melody_win(void) {
    // Fanfarria corta
    play_note(523, 150);
    play_note(523, 150);
    play_note(523, 150);
    play_note(659, 400);
}

void melody_fail(void) {
    // Descenso grave
    play_note(200, 150);
    play_note(150, 150);
    play_note(100, 300);
}

// — Procesar comando recibido —————
void process_command(unsigned char cmd) {
    switch (cmd) {
        case 0: break; // silencio
        case 1: play_note(FREQ_GREEN, NOTE_DUR); break;
        case 2: play_note(FREQ_RED, NOTE_DUR); break;
        case 3: play_note(FREQ_BLUE, NOTE_DUR); break;
        case 4: play_note(FREQ_YELLOW, NOTE_DUR); break;
        case 5: melody_start(); break;
        case 6: melody_win(); break;
        case 7: melody_fail(); break;
    }
}

// — main —————
void main(void) {
    unsigned char prev_strobe = 0;
    unsigned char cmd = 0;

    // Configurar PORTB como entrada
    TRISB = 0xFF;
    TRISC = 0x00; // RC2 salida (PWM buzzer)

    // Inicializar librería Sound (usa Timer2 + CCP1 en RC2)
    Sound_Init(&PORTC, 2); // RC2

    while (1) {
        unsigned char strobe_now = STROBE_PIN;

        // Detectar flanco ascendente del strobe
        if (strobe_now && !prev_strobe) {
            cmd = read_bus();
            process_command(cmd);
        }

        prev_strobe = strobe_now;
    }
}

```

Nota MikroC: Sound_Init y Sound_Play son de la librería Sound integrada en MikroC for PIC. En Project > Library Manager asegúrate de tener activado Sound. El pin RC2 es el CCP1 del PIC16F887 que genera la señal PWM para el buzzer.

Parte 6 — Estructura del repositorio GitHub

```
/
├── ATmega-328P/
│   ├── src/
│   │   ├── main.c
│   │   └── platformio.ini
│   └── PIC16F887/
│       └── simon_pic.c
├── Proteus/
│   └── simon_says.pdsprj
├── Video/
│   └── demo_simulacion.mp4
└── simon_says_informe.pdf
```

Parte 7 — Checklist de entregables vs criterios de evaluación

Criterio	Qué necesitas tener	Puntos
Simulación en Proteus	Circuito completo corriendo	0.75
Control matriz LED	MAX7219 + patrones de cuadrantes + L1/L2/L3	1.0
Comunicación ATmega↔PIC	Bus 4 líneas + tabla de comandos funcional	2.0
Melodías PIC	4 notas + 3 melodías de evento	1.5
Pulsadores	4 botones con debounce + timeout	0.75
3 niveles	Velocidad + longitud de secuencia cambian	1.0
GitHub	1 commit por integrante mínimo	1.5
Informe PDF	Portada + descripción + capturas + código	0.75
Video	Demo de la simulación funcionando	0.75